Access to everything. Now 30% off. Upgrade now

Cpp26 + | C Plus Plus Language + | Technology + | Programming ✓ | Coding +

C++26 `std::function_ref`: A Simple Way to Pass Callbacks

A simple look at `std::function_ref`, how it works, and when to use it instead of `std::function`.

7 min read · 14 hours ago

Sagar Following

Listen Share ••• More



Passing a function to another function is something we do all the time in C++. A retry function is a simple example. You give it some work to do, and it calls that work until it succeeds or runs out of retries.

There are already a few ways to pass that callback in C++. You can use a function pointer, a template, or `std::function`. Each approach has its own limitations.

C++26 adds `std::function_ref` to the mix.

`std::function_ref` **is a non-owning reference to a callable**. It doesn't take ownership of the callback and doesn't need to allocate memory for it.

In this article, we'll look at how `std::function_ref` works, why it was added, and when you would use it instead of `std::function` or the newer `std::copyable_function`.

. . .

Our Traditional Ways So Far

Let's say we have a `retry` function that calls some action a few times until it succeeds. Before `std::function_ref`, there were a few common ways to pass that action to the function.

Function pointer

One option is a function pointer:

```
#include <optional>
#include <string>

using payload = std::optional<std::string>;

payload retry(size_t times, payload (*action)()) {
    for (size_t i = 0; i < times; ++i) {
        if (auto result = action()) {
            return result;
        }
    }

    return std::nullopt;
}
```

A function pointer is simple and fast. There is no allocation involved.
The problem is that it can't hold any state. For example, suppose we have a lambda that keeps track of how many attempts we've made:

```
int attempts = 0;

auto action = [&]() -> payload {
    ++attempts;

    if (attempts < 3)
        return std::nullopt;

    return "Success";
};

retry(5, action); // Does not work
```

This doesn't work because the lambda captures `attempts`. A stateful lambda cannot be passed where a regular function pointer is expected.

Template

Another option is to make `retry` a function template:

```
#include <optional>
#include <string>

using payload = std::optional<std::string>;

template <class F>
payload retry(size_t times, F&& action) {
    for (size_t i = 0; i < times; ++i) {
        if (auto result = action()) {
            return result;
        }
    }

    return std::nullopt;
}
```

Now the same lambda works:

```
int attempts = 0;

auto action = [&]() -> payload {
    ++attempts;

    if (attempts < 3)
        return std::nullopt;

    return "Success";
};

auto result = retry(5, action);
```

This works well with lambdas, including lambdas that capture state.
The downside is that the implementation usually needs to be in a header. If you change the implementation of `retry`, files that use it may need to be recompiled.

std::function

The third option is `std::function`:

```
#include <functional>
#include <optional>
#include <string>

using payload = std::optional<std::string>;

payload retry(size_t times, std::function<payload()> action) {
    for (size_t i = 0; i < times; ++i) {
        if (auto result = action()) {
            return result;
        }
    }

    return std::nullopt;
}
```

The same stateful lambda works here too:

```
int attempts = 0;

auto action = [&]() -> payload {
    ++attempts;

    if (attempts < 3)
        return std::nullopt;

    return "Success";
};

auto result = retry(5, action);
```

This is more flexible than a function pointer, and the implementation of `retry` can stay in a `.cpp` file. But `std::function` is an **owning** wrapper. Depending on the callable, it may need storage and possibly a heap allocation.

There is also a const-correctness issue with `std::function`. A `const std::function` can call a non-const callable, which means the callable can still modify its captured state.

So all three approaches work, but each has something you have to deal with.

This is where `std::function_ref` becomes useful.

. . .

C++26 `std::function_ref`

`std::function_ref` (P0792R14) is a non-owning wrapper.
Think of it as `std::string_view`, but for callables.
Because it's just a **reference**, it *never* allocates heap memory.
It doesn't care if you pass it a function pointer, a lambda, or a custom class—as long as it can be called, `std::function_ref` will point to it.

Furthermore, `std::function_ref` fully supports `noexcept` and `const` in its type signature, allowing you to be strict about what callbacks you accept.



Here is what the code looks like:

```
#include <functional>
#include <optional>
#include <iostream>

using payload = std::optional<std::string>;

// Our modern retry function!
// It takes any callable object without template bloat or heap allocations.
payload retry(size_t times, std::function_ref<payload()> action) {
    for (size_t i = 0; i < times; ++i) {
        if (payload result = action()) {
            return result; // Success!
        }
    }
    return std::nullopt; // Failed after all retries
}

// Example showing noexcept enforcement:
// The compiler will reject any lambda/function that might throw an exception.
void count_calls(std::function_ref<void() noexcept> action) {
    action();
}

int main() {
    int attempts = 0;

    // A stateful lambda capturing 'attempts'
    auto download_action = [&]() -> payload {
        attempts++;
        if (attempts < 3) return std::nullopt;
        return "Data downloaded!";
    };

    // We pass our stateful lambda to retry using function_ref
    payload result = retry(5, download_action);

    if (result) {
        std::cout << "Success: " << *result << "\n";
    }

    return 0;
}
```

> *The Safety Trade-off:*
> *Because* `std::function_ref` *is non-owning, the caller must ensure* `download_action` *outlives the* `retry` *call—* `function_ref` *does not extend lifetimes. Also, unlike* `std::function`*, it cannot be null or default-constructed.*
> *It must always refer to a valid callable, meaning you never need to check it with* `operator bool`*.*

Note:
`std::function_ref` *uses* `std::invoke` *semantics internally, but unlike* `std::function`*, it does not accept a raw member-function pointer directly because that would bloat its size. For member functions, use the zero-overhead* `std::nontype<&Class::method>` *syntax!)*

. . .

How does it work?

Under the hood, `std::function_ref` usually stores exactly two things:

1. **A pointer to the object** (the "bound entity" — e.g., your lambda's captured data).
2. **A function pointer** (a "thunk" that knows how to invoke your specific object).

Because it only stores two pointers, it easily fits into CPU registers. Every specialization of `function_ref` is trivially copyable, meaning it passes efficiently in registers.



. . .

What if you need to store the callback?

`std::function_ref` is perfect for synchronous calls. But what if you are writing an asynchronous system or a GUI framework where the callback must be stored in a class to be executed later? You can't use a reference for that, or it will dangle.

For those cases, C++ introduced new owning counterparts to `std::function_ref`:

- `std::copyable_function`: Use this when your stored callback needs to be copied. It officially replaces the outdated `std::function` by properly respecting `const` and `noexcept` qualifiers on your signatures, preventing the infamous constness-bugs of the past.
- `std::move_only_function`: Use this when your callback captures move-only types (like `std::unique_ptr`) and cannot be copied.

Rule of thumb:

- Need to store a callback for later? Use `std::copyable_function` or `std::move_only_function`.
- Just borrowing a callback for the duration of a function call? Use `std::function_ref`.

. . .

The 2026 Optimization: Flattening "Double Indirection"

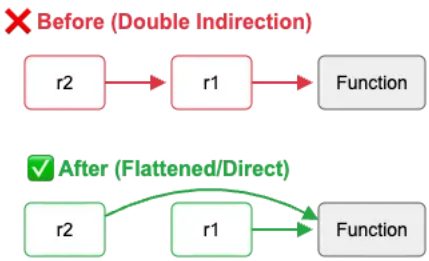
As `std::function_ref` was finalized, the C++ committee noticed an opportunity to improve its codegen (documented in P3961R1, March 2026).

Just like regular core-language function pointers, it is perfectly valid to assign a strictly `noexcept` callable to a non-noexcept `function_ref`. However, imagine assigning a `function_ref` to *another* `function_ref` with a wider signature. Initially, the standard

treated the first reference just like any other callable object. This meant the second `function_ref` pointed to the *first* `function_ref`, which pointed to the ***actual*** function.

This created a **double indirection.** Not only is this slightly slower, but it creates a nasty bug: if the first `function_ref` goes out of scope, the second one ends up pointing to garbage!

To fix this issue, the standard was updated so that if you initialize a `function_ref` from another compatible `function_ref`, it copies the underlying pointers directly. No more double indirection!



Here is how this looks in code now:

```
void fast_task() noexcept;

// We create a function_ref that promises not to throw
std::function_ref<void() noexcept> r1 = fast_task;

// Mirroring core-language function pointer behavior, this conversion is allowed.
// Today, r2 directly copies the internal pointers of r1, flattening the reference!
std::function_ref<void()> r2 = r1;

r2(); // Calls fast_task() directly, bypassing r1 entirely.
```

By safely unwrapping these references at compile time, the compiler avoids unnecessary jumps and saves you from weird runtime bugs if `r1` is later destroyed.

. . .

Show Me the Numbers (Benchmark)

If you're skeptical about the "zero-overhead" claim, here is a conceptual look at how `std::function_ref` stacks up against `std::function` when executing a simple stateful lambda millions of times:

```
auto start = std::chrono::high_resolution_clock::now();
for (int i = 0; i < 10'000'000; ++i) {
    retry(3, [&] { return std::optional{"ok"}; });
}
```

- `std::function_ref`: Executes nearly as fast as a raw C function pointer (often entirely inlined by the compiler).
- `std::function`: Substantially slower due to constructor overhead, heap allocation checks (or actual heap allocations if the capture is large enough), and heavier underlying mechanics preventing the compiler from inlining effectively.

. . .

Closing Thoughts

`std::function_ref` is a simple way to pass a callable without taking ownership of it.

- It **doesn't allocate** memory for the callable.

- It is a small wrapper that typically contains two pointers.
- It **doesn't require** making the function a template just to accept a callback.
- It **cannot be null or default-constructed.** It always refers to a valid callable.
- **P3961R1** also avoids an extra level of indirection when one compatible `function_ref` is created from another.

Compiler Support: As of late 2026, GCC, Clang, and MSVC have implementations of `std::function_ref`, based on **P0792R14** and its later updates.

So if you have a function that only needs to use a callback during the current call, `std::function_ref` is worth considering instead of `std::function`.

. . .

Found this article helpful? Please **Clap** 👏 and **follow** for more C++ and system programming content.

Cpp26 + | C Plus Plus Language + | Technology + | Programming ✓ | Coding +

Open in app ↗

Medium



Following

Written by Sagar

342 followers · 7 following

System Programmer | C++ | Linux System Programming | Rust | Python | Cloud Computing | DevOps | Low Latency Backends